Backend Explanation

Ghost Kitchen API · Phase 2

1. System Architecture & Request Lifecycle

The backend is a layered MVC-style application built on Node.js and Express. Source code lives under Phase2/src/ and is split into four responsibility layers.

Server Layer (server.js) — Initializes Express, loads environment variables via dotenv, enables CORS and JSON body parsing, then mounts every route module under the /api/v1 prefix. The server listens on a configurable port (default 3000).

Route Layer (routes/) — Eleven Express routers define the HTTP endpoints. Each route chains authentication and authorization middleware before delegating to the model layer. Route handlers validate input, invoke the appropriate model function, and format the response with a proper status code (200, 201, 400, 401, 403, 404, 409, 500).

Middleware Layer (middleware/) — auth.js extracts the Bearer token, verifies it with jwt.verify(), and attaches the decoded payload to req.user. For courier accounts it also resolves courier_id from the database. rbac.js is a middleware factory: authorize(...roles) returns middleware that checks req.user.role and rejects mismatches with 403.

Model Layer (models/) — Ten modules export async functions that execute raw parameterized SQL via the pg library. Simple reads/writes call pool.query(). Transactions acquire a dedicated client through pool.connect() and manage BEGIN / COMMIT / ROLLBACK manually, always releasing the client in a finally block.

Request Lifecycle

Express receives the HTTP request → JSON body parser extracts the payload → auth middleware verifies the JWT and attaches req.user → RBAC middleware checks role permissions → the route handler validates input → the model function executes SQL → a JSON response is returned with the relevant status code. A failure at any step short-circuits the chain (e.g., expired JWT → 401; wrong role → 403; missing field → 400).

2. Authentication Flow & RBAC Enforcement

Registration — POST /auth/register validates that username, password, and role are present and that the role is one of admin, kitchen_staff, or courier. The password is hashed with bcrypt (10 salt rounds) and inserted into the Users table. Duplicate usernames trigger PostgreSQL error 23505, mapped to 409 Conflict.

Login — POST /auth/login finds the user by username, compares the password against the stored bcrypt hash, and issues a JWT containing user_id, username, and role. The token is signed with a server-side secret from the .env file and expires after 24 hours.

Token Verification — Every protected endpoint passes through the authenticate middleware. It decodes the Bearer token, verifies it, and populates req.user. For courier users it additionally resolves courier_id from the Couriers table and attaches it to req.user so that courier-specific routes can enforce ownership without extra queries.

RBAC Middleware — authorize() accepts a list of allowed roles and returns a middleware that rejects any request whose req.user.role is not in the list (403 Forbidden).

The table below summarizes role permissions across all resources:

Resource	Allowed Roles	Notes
Kitchen Hubs	Admin	Full CRUD
Virtual Brands	Admin	Full CRUD
Inventory	Admin, Kitchen Staff	KS cannot DELETE
Allocations (read)	Admin, Kitchen Staff	Read-only for KS
Allocations (write)	Admin	Create / Update / Delete
Menu Items	Admin, Kitchen Staff	KS cannot DELETE
Orders	Admin (CUD), Admin+KS (R)	Courier: single GET only
Couriers	Admin, Courier (own)	Courier sees own profile
Deliveries	Admin, Courier (own)	Courier completes own
Payouts	Admin, Courier (/my)	Courier: own payouts only
Reports (x3)	Admin	Sales, inventory, courier perf.

Courier Data Isolation — Beyond role checks, courier endpoints enforce ownership at the handler level. Routes filter by req.user.courier_id and return 403 if a courier tries to read or modify another courier's deliveries, profile, or payouts.

3. Transaction Flows & Rollback Conditions

Two transaction scenarios from the Phase 1 ACID documentation are implemented using actual BEGIN / COMMIT / ROLLBACK on dedicated connections. Both follow a try-catch-finally pattern with client.release() in finally to prevent leaks.

Transaction 1 — Order Placement with Inventory Deduction

Implemented in order.model.js → placeOrder(). The dedicated client begins a transaction, inserts a new order with status PLACED, then iterates over each requested item. For every item it inserts into OrderItems — which fires the database trigger trg_check_brand_inventory_allocation to validate that the brand has not exceeded its allocated inventory share. Next it updates Inventory to deduct the requested quantity, verifying that total_quantity >= requested amount (otherwise it throws an error). On success the transaction commits; on any failure the catch block issues ROLLBACK.

Rollback causes: brand allocation exceeded (trigger exception), insufficient inventory (zero rows updated), FK violation (invalid item or brand), or any unexpected database error.

Transaction 2 — Delivery Completion & Payout Processing

Implemented in delivery.model.js → completeDelivery(). The transaction first selects and validates the delivery (exists, not already completed, correct courier). It then updates the delivery status to COMPLETED, the order status to DELIVERED (the trigger trg_enforce_order_status_transition validates the state transition), and sets the courier back to AVAILABLE. It calculates the payout as 15% of the order total (SUM of unit price × quantity from OrderItems joined with MenuItems) and inserts a Payouts record. Finally it commits.

Rollback causes: delivery not found or already completed; order-status trigger rejects transition (order already DELIVERED or CANCELLED); courier not authorized for the delivery; payout insertion failure.

Role of Database Triggers in Transactions

Both triggers operate inside the transaction boundary. When a trigger raises an exception, PostgreSQL automatically aborts the statement; the application's catch block then issues ROLLBACK to release locks. This provides defense-in-depth: even if the application layer misses a validation, the database enforces consistency.

4. Raw SQL & Connection Pooling Strategy

Raw SQL — The project requirements prohibit ORM-only implementations. Every query in every model file is hand-written parameterized SQL executed via the pg library (e.g. "SELECT * FROM Orders WHERE order_id = $1" with a parameter array). This gives full control over query structure and direct access to PostgreSQL-specific features such as triggers, views, and CHECK constraints.

Parameterized Queries — All queries use positional parameters ($1, $2, …) rather than string concatenation, preventing SQL injection. User input is never interpolated into query strings.

Connection Pooling — pg.Pool maintains reusable connections configured via environment variables (DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD). Simple queries call pool.query(), which handles checkout and return automatically. Transactions call pool.connect() for a dedicated client whose lifecycle is managed manually: BEGIN → queries → COMMIT or ROLLBACK → client.release() in finally. This is necessary because transaction state is tied to a specific connection.

5. Key Design Decisions & Tradeoffs

Flat Architecture — Each model file exports its own functions instead of inheriting from a generic CRUD base class. This favors readability over DRY, appropriate at this project's scale.

No Validation Library — Input validation relies on simple if-checks in routes combined with database-level constraints (CHECK, NOT NULL, UNIQUE, FK). The database acts as the ultimate safety net.

Courier–User Linking — A user_id FK column was added to Couriers so courier accounts can authenticate and access their own data — the minimal schema change from Phase 1.

Payout Calculation — Payouts are computed as 15% of the order total inside the transaction, ensuring the amount is consistent with the order data at commit time.

Error Mapping — PostgreSQL error codes are mapped to HTTP status codes (23505 → 409 Conflict; trigger exception → 400 Bad Request). This gives clients useful feedback without exposing internal database details.

JWT over Sessions — Stateless JWT authentication avoids server-side session storage. Tokens cannot be revoked before expiry, but the simplicity tradeoff is worthwhile for this project's scope.

API Versioning — All endpoints live under /api/v1, following REST convention and allowing future versions to coexist without breaking existing clients.